

When can you differentiate coupled simulation components separately?

A cheap test, and a cold plate that answers it

Tesseract Hackathon 2026 · Track: multi-physics & coupled systems · Apache-2.0 · every number below is printed by a named script in this repository

Abstract

Composing differentiable simulation components across a boundary is expensive: it demands that each component expose derivatives, and that the composition solve an implicit system rather than a chain. A practitioner facing that cost reasonably asks whether the shortcut — differentiating each component in isolation and multiplying the pieces together — is good enough.

We build a natural-convection cold plate from four Tesseracts in four languages with four different differentiation strategies, and use it to answer that question quantitatively. Three findings. **First**, the components are genuinely interchangeable: replacing a JAX-autodiffed solver with a Fortran one differentiated by an Enzyme compiler pass leaves the end-to-end gradient unchanged to 5.3×10^{-12} . **Second**, the cost of skipping composition is wildly regime-dependent — from 0.01% to 86% relative error on the same code — and nothing in the forward solution indicates which regime you are in. **Third**, the obvious diagnostic is wrong and a better one exists: the spectral radius of the fixed-point Jacobian, $\rho(\Phi_T)$, cannot predict the error (we exhibit a case where it is constant while the error varies 136-fold), whereas the *directional* gain $\gamma = \|\Phi_T^T g\|/\|g\|$ — one vector-Jacobian product — tracks it.

1. The problem

Two solvers that feed each other do not form a pipeline. In our case a Stokes–Brinkman flow solver and an advection–diffusion thermal solver are coupled in both directions: buoyancy makes temperature drive the flow, advection makes the flow drive temperature. The steady state is a fixed point,

$$T^* = \Phi(T^*, \theta), \Phi = \text{thermal}(\text{fluid}(T)),$$

and the sensitivity of any objective $J(T^*)$ requires the implicit function theorem,

$$dJ/d\theta = (\partial\Phi/\partial\theta)^T \lambda, (I - \Phi_T)^T \lambda = g, g = dJ/dT. \quad (1)$$

The shortcut treats the loop as feed-forward, keeping only the first term of

$$\lambda = g + \Phi_T^T g + (\Phi_T^T)^2 g + \dots \quad (2)$$

Whether that is acceptable turns out to be an empirical question with a theoretical answer.

Relation to existing work

Almost everything here has been done before, and better, in one respect or another. Differentiable topology optimisation of thermo-fluidic devices: **TOFLUX** (Padmanabha et al., arXiv:2508.17564, 2025) is an open-source JAX framework covering thermo-fluidic coupling, FSI and non-Newtonian flow. Natural-convection heat-sink design: **Alexandersen et al.** (*Int. J. Heat Mass Transfer*, 2016, arXiv:1508.04596) solve the 3D Boussinesq problem with full Navier–Stokes at 40–330 million degrees of freedom; this work is 2D, Stokes and 96×96 , and the branching structure it finds reproduces theirs qualitatively rather than adding to it. Sensitivity of fixed points under approximate linearisation: **Padway and Mavriplis** (*Numerical Algorithms*, 2021, arXiv:2104.02826) analyse tangent and adjoint problems linearised about non-stationary points; truncating (2) is a severe special case of an approximate adjoint, and that $\Phi_T^T g$ leads the error follows directly from the expansion.

Three things are left. **Composition across genuinely heterogeneous components**, which those frameworks deliberately avoid — TOFLUX is one framework in one process because that is the sane way to build a framework — whereas here a hand-adjointed C++ solver, a compiler-differentiated Fortran solver, a JAX solver and a PyTorch model compose into one function, two of them interchangeably. **The refutation of the spectral radius**, which we have not seen stated: $\rho(\Phi_T)$ is the diagnostic a practitioner reaches for first and Section 5 shows it constant while the error moves 136-fold. And **γ as an operational check** — the analysis is standard, but making it a single VJP that runs as a pipeline assertion, and measuring what it actually predicts, is not.

2. The composition

Four Tesseracts, four languages, four ways of producing a derivative:

component	language	derivatives from
stokes_brinkman	C++ / Eigen	hand-derived discrete adjoint (no AD tool)
thermal_advdiff	JAX	jax.jvp / jax.vjp
thermal_fortran	Fortran	Enzyme, an LLVM compiler pass
material_map	PyTorch	torch.autograd

The fluid system is linear in $w = (u, v, p)$, so $A(\alpha) w = b(T)$ and its exact JVP and VJP are extra solves against the same sparse LU — a transpose solve plus an analytic scatter, by hand. The Fortran component inverts that approach: flang emits LLVM IR, an Enzyme pass differentiates it, and $\partial R/\partial T$ is recovered *exactly* from nine Enzyme JVPs by a 3×3 colouring (the stencil is five-point, so cells whose indices agree mod 3 never interact).

Both halves of the gradient are Krylov solves whose matvecs cross the container boundary: Newton-Krylov forward, applying $(\Phi_T - I)$ via JVPs, and GMRES for the adjoint of (1), applying $(I - \Phi_T)^T$ via VJPs. At our operating point the loop gain exceeds one, so Picard iteration provably cannot converge — Newton needs the linearisation invertible, not contractive. The JVP endpoints are therefore not a convenience; they are what makes the forward problem solvable.

Chain versus loop. The distinction matters and is often blurred. A *chain* — A feeds B feeds an objective — is differentiable by one sweep of the chain rule; a single `jax.grad` over wrapped components suffices and nothing is solved. This is a *loop*: the fluid solver's output is the thermal solver's input and vice versa, so no ordering makes one sweep sufficient. The steady state must be solved for, and its sensitivity requires a second, transposed solve whose operator exists only as an action realised by calling both components.

Why not `jax.custom_vjp`? The fair objection to any containerised differentiable pipeline is that one could attach a hand-written backward pass to a component and keep everything in one process. That is correct when the components are Python. Here the components are C++/Eigen and flang/Enzyme-built Fortran, whose build toolchains (LLVM 19, flang) do not belong inside a JAX process; the boundary is crossed thousands of times per solve in both directions, which is a conversation rather than a wrapped call; and because the contract is a schema, the two thermal implementations are substitutable with no change to the caller, which a `custom_vjp` written against one implementation is not.

Two claims are enforced by the build rather than asserted: neither the C++ nor the Fortran image can import `jax`, `torch`, `tensorflow`, `autograd` or `casadi`, and the Fortran library must contain `cosh` among its linked symbols — a function present in no source file, being Enzyme's generated derivative of the `tanh` in the Péclet weighting. Both are re-checkable from outside the build with `scripts/verify_integrity.sh`.

Components are interchangeable. `thermal_advdiff` and `thermal_fortran` implement the same equation behind the same schema. Swapping them (`compare_thermal_backends.py`) changes the component field by 7.1×10^{-16} , the JVP by 1.4×10^{-15} , the VJP by 4.3×10^{-15} , the converged coupled state by 4.8×10^{-12} , and **the end-to-end gradient by 5.3×10^{-12}** , cosine 1.000000000000. The gradient does not depend on which derivative technology produced it.

3. Validation

Classical critical Rayleigh number. Stokes flow is the infinite-Prandtl limit, which is the regime in which the classical onset result $Ra_c = 1707.762$ is derived, so it is the correct benchmark for this solver. (A Navier-Stokes benchmark such as de Vahl Davis is not: at $Pr = 0.71$ the inertia term we omit is not small, and disagreement would prove nothing.) At the conduction state the coupling loop is the linear stability operator, so onset occurs exactly where $\rho(\Phi_T) = 1$. Bisection on `Ra (benchmark_critical_rayleigh.py)`:

aspect ratio	1	2	4	8
Ra_c measured	2519.07	1970.84	1779.02	1707.97
excess	+47.5%	+15.4%	+4.2%	+0.012%

No-slip side walls stabilise a confined layer, so Ra_c must exceed the unbounded value and descend towards it as the box widens. It does, agreeing to four significant figures. One number checks the fluid solver, the thermal solver, the coupling between them and the loop-gain machinery simultaneously.

Order of accuracy. Richardson extrapolation on two independent grid trios gives observed order 1.87 and 1.83, with 0.02-0.05% error on the finest grid (`grid_convergence.py`).

Independent implementations agree. The composed pipeline reproduces a separately written monolithic reference to 1.5×10^{-12} on the converged coupled state, reached by a different nonlinear solver (5 Newton iterations against 21 Picard). Conduction-only energy balance closes to 5.3×10^{-15} .

The composed gradient is exact. A directional derivative agrees with central differences to 8.3×10^{-6} , holding at $\sim 10^{-5}$ across step sizes from 10^{-3} to 10^{-4} — that plateau is the differencing noise floor, since the fixed point is converged only to $\sim 10^{-10}$.

4. What the shortcut costs

We compare against the *charitable* shortcut: it uses the fluid solver's adjoint in full and differentiates the whole chain $\rho \rightarrow \alpha \rightarrow (u, v) \rightarrow T$, and is exact except that it treats the temperature entering buoyancy as constant. It errs only by ignoring the loop.

At a strongly coupled state ($Ra = 3 \times 10^4$) it carries **86% relative error**, cosine 0.53 against the true gradient, and the **wrong sign on 33% of design variables**. At the states our optimiser actually visits it is 4-20% off with cosine above 0.98. Same code, same physics, two orders of magnitude difference — and J moves by only 13% across the range over which the gradient error grows from 3×10^{-6} to 0.86. **The forward solution gives no warning.**

Spatially, the error is not noise. At high coupling the exact gradient develops a coherent region of *positive* sensitivity — adding material there makes the chip hotter, because it obstructs the convection cell removing the heat — and the shortcut has no such region anywhere. The disagreement is one contiguous blob: an entire physical term, missing.

5. Predicting it

The obvious statistic fails. $\rho(\Phi_T)$ is the natural candidate and correlates respectably (0.825 with log error) when designs and Rayleigh numbers are varied together. But it is a worst case over all directions, and gain along directions the objective never excites costs nothing. Two states at the same Ra have $\rho = 0.682$ with 40% error and $\rho = 0.377$ with 79% error: it orders them backwards.

A constant cannot explain a variable. The decisive experiment holds the design, the Rayleigh number and hence the entire coupled state fixed, and varies only the objective. $\rho(\Phi_T)$ is then *identical by construction*, while $g = dJ/dT$ differs (`objective_sweep.py`):

objective	outlet	top-half	chip peak	chip mean	domain	left column
γ	0.0027	0.0076	0.0261	0.0300	0.0967	0.3879
naive error	0.0117	0.0026	0.0385	0.0399	0.0211	0.3535

$\rho(\Phi_T) = 0.5481$ on every column while the error varies **136-fold**. This refutes the spectral radius outright; no further data can rescue it.

The directional gain. Equation (2) says the leading error is $\Phi_T^T g$, so the natural relative measure is

$$\gamma = \|\Phi_T^T g\| / \|g\|, \quad (3)$$

one VJP through the loop. Across four design families and five Rayleigh numbers, log γ correlates with log error at **0.995**, against 0.825 for ρ (`predict_error.py`), and empirically $\text{error} \approx \gamma$ while γ is small.

γ is a guide, not a formula. Across objectives it correlates at 0.80 and misses two rows above by about $4\times$ in each direction — expected, since γ bounds the leading error in λ whereas the quantity of interest also depends on how Φ_{θ^T} maps that error into design space. As an order-of-magnitude test it is serviceable: below $\gamma \approx 0.01$ the shortcut costs about a percent; above $\gamma \approx 0.1$ it is not worth having. It ships as `coupling_check.py`, which requires only a JAX-traceable loop.

6. The design problem

Minimising mean chip temperature subject to a 35% solid-material budget, with density filtering and Heaviside continuation, 120 iterations at 96×96 : **J falls from 8.18 to 1.26, an 84.6% reduction**. The optimiser finds a branching tree that conducts heat toward the sink while leaving channels open for buoyancy to carry the remainder — the structure the natural-convection topology optimisation literature reports.

7. What we do not claim

We ran that optimisation twice, once with each gradient, and **both succeeded**; the shortcut finished marginally lower (1.2576 against 1.2588). We are not going to dress that up. An optimiser is not evidence that a gradient is right: Adam normalises per coordinate and therefore consumes only direction, and along this trajectory the shortcut's cosine stays above 0.98. A wrong gradient can be a serviceable search direction and a useless sensitivity at the same time. The failure matters when the gradient is used *as a quantity* — sensitivity analysis, uncertainty propagation, deciding which variables matter.

Further limitations, stated plainly. The physics is two-dimensional and Stokes, so inertia is absent; that is what makes the infinite-Prandtl benchmark appropriate and a finite-Prandtl one inappropriate. The optimisation runs at $Ra = 10^3$ because near-uniform intermediate densities — where topology optimisation must start — have no reachable steady state above roughly that, open-cavity convection there being genuinely unsteady. γ has been tested on one physical system with two coupling structures; that it transfers is a conjecture supported by its derivation, not a demonstrated fact.

8. Reproducibility

Four containerised components with dependencies pinned to the versions that produced these numbers; 20 tests running in CI without Docker; one script per claim (`compare_thermal_backends.py`, `benchmark_critical_rayleigh.py`, `grid_convergence.py`, `validate_pipeline.py`, `sweep_coupling.py`, `predict_error.py`, `objective_sweep.py`, `optimize.py`); and every headline number reproduced through *either* thermal backend (`scripts/validate_both_backends.sh`), agreeing to nine or ten significant figures. The integrity claims of Section 2 are re-checkable against the built images with `scripts/verify_integrity.sh`.